

CAUCASUS UNIVERSITY · SCHOOL OF TECHNOLOGY

Python Programming II

Meeting 4 · Recursion, Modules & Tuples

palindromes · GCD · Fibonacci · modules · files · tuples

Lecturer: **Prof. Maksim Iavich**

miavich@cu.edu.ge · Scientific Cyber Security Association



MEETING 4

More Recursion, Modules & Tuples

Reusable code across files, and the first structured data type

Task 1

Recursive palindrome check

- Write a function isPal(s) that returns True if the list/string s is a palindrome.
- A single character (or empty) is a palindrome; otherwise the first and last must match and the middle must also be a palindrome.

Libraries / focus: *Recursion + slicing*

Solution — isPal

```
def isPal(s):
    if len(s) <= 1:
        return True
    else:
        return s[0] == s[-1] and isPal(s[1:-1])

return isPal(toChars(s))
```

Task

Greatest common divisor — iterative

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Greatest common divisor — iterative

```
def gcdIter(a, b):
    '''
    a, b: positive integers
    returns: a positive integer, the greatest common divisor of a & b.
    '''
    c = min(a, b)
    while c > 0:
        if (a % c) == 0 and (b % c) == 0:
            return c
        else:
            c -= 1
```

Task

Greatest common divisor — recursive (Euclid)

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Greatest common divisor — recursive (Euclid)

```
def gcdRecur(a, b):
    # Base case is when b = 0
    if b == 0:
        return a
    # Recursive case
    return gcdRecur(b, a % b)
```

Euclid's algorithm — the idea

- Example: a = 64, b = 48.
- 64 = 48 × 1 + 16 → remainder 16
- 48 = 16 × 3 + 0 → remainder 0
- **When the remainder becomes 0, the last divisor (16) is the GCD.**
- Recursively: gcd(a, b) = gcd(b, a % b), with base case gcd(a, 0) = a.

Recursion with multiple base cases — Fibonacci

- **Leonardo of Pisa (aka Fibonacci) modelled the following challenge:**
  - A newborn pair of rabbits (one female, one male) is put in a pen.
  - Rabbits mate at the age of one month; gestation is one month.
  - Assume rabbits never die and every female produces one new pair each month from its second month on.
  - How many female rabbits are there at the end of one year?

Fibonacci — base & recursive cases

- After month 0 → 1 female. After month 1 → still 1 female (now pregnant).
- In general: females(n) = females(n-1) + females(n-2).
- **Base cases: females(0) = 1, females(1) = 1**
- **Recursive case: females(n) = females(n-1) + females(n-2)**
- Sequence: 1, 1, 2, 3, 5, 8, 13, …

Task

Fibonacci — recursive code

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Fibonacci — recursive code

```
def fib(x):
    """assumes x an int >= 0
       returns Fibonacci of x"""
    if x == 0 or x == 1:
        return 1
    else:
        return fib(x-1) + fib(x-2)
```



MODULES

Modules

Splitting code across reusable files

Task

Example module — circle.py

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Example module — circle.py

```
# the file circle.py contains:
pi = 3.14159

def area(radius):
    return pi*(radius**2)

def circumference(radius):
    return 2*pi*radius
```

Task

Importing and using a module

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Importing and using a module

```
import circle
pi = 3
print(pi)                      # 3
print(circle.pi)               # 3.14159
print(circle.area(3))          # 28.27431
print(circle.circumference(3))  # 18.849539999999998
```

Task

Other importing styles

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Other importing styles

```
# if we don't want to prefix names with the module,
# and the names don't collide with other bindings:
from circle import *
print(pi)
print(area(3))
```

Task

Files — writing to a file

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Files — writing to a file

```
nameHandle = open('kids', 'w')
for i in range(2):
    name = input('Enter name: ')
    nameHandle.write(name + '\n')
nameHandle.close()
```



TUPLES

Tuples

Ordered, immutable sequences

Tuples

- An ordered sequence of elements, can mix element types.
- **immutable — cannot change element values.**
- Represented with parentheses: te = () ; t = (2, "one", 3)
- t[0] → 2 ; (2,"one",3) + (5,6) → (2,"one",3,5,6)
- t[1:2] → ("one",) ; t[1:3] → ("one",3)
- t[1] = 4 → error, can't modify the object

Tuples — swapping variables

- **Conveniently used to swap variable values.**
  - Wrong: x = y ; y = x   (loses the old x)
  - Works: temp = x ; x = y ; y = temp
  - Pythonic: (x, y) = (y, x)

Task

Returning multiple values with a tuple

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Returning multiple values with a tuple

```
def quotient_and_remainder(x, y):
    q = x//y
    r = x%y
    return (q, r)

(quot, rem) = quotient_and_remainder(4, 5)
```

Task

Iterating over tuples — get_data

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Iterating over tuples — get_data

```
def get_data(aTuple):
    nums = ()
    words = ()
    for t in aTuple:
        nums = nums + (t[0],)
        if t[1] not in words:
            words = words + (t[1],)
    min_nums = min(nums)
    max_nums = max(nums)
    unique_words = len(words)
    return (min_nums, max_nums, unique_words)
```

Task 2

oddTuples

- Write a procedure called oddTuples, which takes a tuple as input and returns a new tuple as output, where every other element of the input tuple is copied, starting with the first one.
- So if test is the tuple ('I', 'am', 'a', 'test', 'tuple'), then evaluating oddTuples on this input would return the tuple ('I', 'a', 'tuple').

Libraries / focus: *Tuples — immutability and iteration*

Solution — oddTuples

```
def oddTuples(aTup):
    i = 0
    newTup = ()
    for index in aTup:
        if i%2 == 0:
            newTup += (index,)
        i += 1
    return newTup

# Test Code
oddTuples((2, 2, 15, 18, 20))
```

Task 3

★ NEW

Open a link with the webbrowser module

- The standard-library webbrowser module can open URLs in the user's default browser.
- Write a helper open_report(url) that opens a security report page automatically after a scan finishes.

Libraries / focus: *webbrowser — a standard-library module (ties into Project SentinelAI)*

Solution — using the webbrowser module

★ NEW

```
import webbrowser

def open_report(url):
    """Open a report URL in the default browser."""
    print('Opening report:', url)
    webbrowser.open(url)

open_report('https://cu.edu.ge')
```

Task 4

★ NEW

Store scan results as immutable tuples

- A network scan produces records like (ip, port, service).
- Store several results in a list of tuples, then loop over them to print only the open web ports (80 or 443).
- Tuples are perfect here: a finished scan record should not change.

Libraries / focus: *Tuples — immutable records for scan data*

Solution — iterating scan records

★ NEW

```
scan = [
    ('10.0.0.5', 22, 'ssh'),
    ('10.0.0.5', 80, 'http'),
    ('10.0.0.7', 443, 'https'),
    ('10.0.0.7', 3306, 'mysql'),
]

for (ip, port, service) in scan:
    if port in (80, 443):
        print('Web service on', ip, '->', service)
```



END OF MEETING 4

Recap

Recursion, reusable modules, and immutable tuple records